Transformers

SISEPUEDE includes a predefined library of **transformations** that can be customized and combined to create detailed and comprehensize strategies–based on expected outcomes–for reducing emissions of greenhouse gasses at scale.

Transformations are outcomes that can be acheieved through the implementation of one or more policies. For example, mode shifting transportation can be achieved through a number of potential policies, including urban planning, taxes on fuels, reductions of public transportation fares, construction of infrastructure, and more.

Transformations are defined in *Transformation* classes in python. Collections of transformations are called **strategies** (link).

Transformers vs. Transformations

SISEPUEDE uses simlar terms to refer to two different components of the framework for modifying

Transformers are pre-defined callable classes in Python that modify a base set of trajectories to reflect a desired outcome. These classes, which include default values, can be called with different functional specifications that are defined in configuration files to allow for flexibility in applying the transformation.

Transformer Conventions

Transformers follow some conventions. Understanding these conventions can help increase readability and interpretation of conventions.

Transformer Codes follow a simple structure

`TFR:SUBSEC_CODE:ACTION_DESCRIPTION`

Actions are generally defined using the following abbreviations:

- *DEC*: Decrease
- *INC*: Increase
- *SHIFT*: Move from one category to another

For example,

Transformers

transformer_id	transformer
0	Baseline
1	AGRC: Improve rice management
2	AGRC: Decrease demand for unhealthy crops
3	AGRC: Decrease Exports
4	AGRC: Reduce supply chain losses
5	AGRC: Expand conservation agriculture
6	AGRC: Improve crop productivity
7	AGRC: Increase residue removal
8	LNDU: Stop deforestation
9	LNDU: Decrease deforestation and increase silvopasture at the same time
10	LNDU: Expand sustainable grazing practices

11	LNDU: Rehabilitate degraded land
12	LNDU: Increase Reforestation
13	LNDU: Expand silvopasture
14	LNDU: Partial land use reallocation
15	LSMM: Increase biogas capture at anaerobic decomposition facilities
16	LSMM: Improve manure management for cattle and pigs
17	LSMM: Improve manure management for other animals
18	LSMM: Improve manure management for poultry
19	LVST: Reduce enteric fermentation
20	LVST: Decrease exports
21	LVST: Increase livestock productivity
22	SOIL: Improve lime application
23	SOIL: Improve fertilizer application
24	TRWW: Increase biogas capture
25	TRWW: Increase septic compliance
26	WALI: Improved industrial wastewater treatment

27	WALI: Improved rural wastewater treatment
28	WALI: Improved urban wastewater treatment
29	WASO: Consumer food waste reduction
30	WASO: Increase composting and biogas
31	WASO: Increase biogas capture
32	WASO: Biogas for energy production
33	WASO: Incineration for energy production
34	WASO: Increase landfilling
35	WASO: Increase recycling
36	CCSQ: Increase direct air capture
37	ENTC: Reduce transmission losses
38	ENTC: Least cost solution
39	ENTC: Clean hydrogen
40	ENTC: 95% of electricity is generated by renewables in 2050
41	FGTV: Minimize leaks
42	FGTV: Maximize flaring
43	INEN: Maximize industrial energy efficiency
44	INEN: Maximize industrial production efficiency
45	INEN: Fuel switch high- and low-temp thermal processes.
46	SCOE: Reduce end-use demand for heat energy by improving building shell
47	SCOE: Increase appliance efficiency
48	SCOE: Switch to electricity for heat using heat pumps, electric stoves, etc.
49	TRDE: Reduce demand for transport
50	TRNS: Increase electric transportation energy efficiency
51	TRNS: Increase non-electric transportation energy efficiency
52	TRNS: Increase occupancy for private vehicles

53	TRNS: Electrify light duty road transport
54	TRNS: Fuel switch maritime
55	TRNS: Fuel switch medium duty road transport
56	TRNS: Electrify rail
57	TRNS: Mode shift freight
58	TRNS: Mode shift passenger vehicles to others
59	TRNS: Mode shift regional passenger travel
60	IPPU: Reduce cement clinker
61	IPPU: Demand management
62	IPPU: Reduce use of HFCs
63	IPPU: Reduce Nitrous Oxide emissions
64	IPPU: Reduce other fluorinated compounds
65	IPPU: Reduce use of PFCs
66	PFLO: Change diets
67	PFLO: Industrial carbon capture and sequestration

class **sisepuede.transformers.transformers.Transformers**(*dict_config: Dict, code_baseline: str = 'TFR:BASE', df_input: DataFrame | None = None, field_region: str | None = None, logger: Logger | None = None, regex_code_structure: Pattern = re.compile('TFR:(\D*):(.*$)'), regex_template_prepend: str = 'sisepuede_run', **kwargs*)

Build collection of Transformers that are used to define transformations.

Includes some information on

Initialization Arguments

- **dict_config: configuration dictionary used to pass parameters to**

  transformations. See ?TransformerEnergy._initialize_parameters() for more information on requirements.

- **dir_jl: location of Julia directory containing Julia environment and**

  support modules

- **fp_nemomod_reference_files: directory housing reference files called by**

  NemoMod when running electricity model. Required to access data in EnergyProduction. Needs the following CSVs:

  - **Required keys or CSVs (without extension):**

    1. CapacityFactor
    2. SpecifiedDemandProfile

Optional Arguments

- **fp_nemomod_temp_sqlite_db: optional file path to use for SQLite database**

  used in Julia NemoMod Electricity model * If None, defaults to a temporary path sql database

- logger: optional logger object
- **model_afolu: optional AFOLU object to pass for property and method access.**
  - **NOTE: if passing, ensure that the ModelAttributes objects used to**

    instantiate the model + what is passed to the model_attributes argument are the same.

- **model_circecon: optional CircularEconomy object to pass for property and**

  method access. * NOTE: if passing, ensure that the ModelAttributes objects used to

  instantiate the model + what is passed to the model_attributes argument are the same.

- **model_enerprod: optional EnergyProduction object to pass for property and**

  method access. * NOTE: if passing, ensure that the ModelAttributes objects used to

  instantiate the model + what is passed to the model_attributes argument are the same.

- **model_ippu: optional IPPU object to pass for property and method access.**
  - **NOTE: if passing, ensure that the ModelAttributes objects used to**

instantiate the model + what is passed to the model_attributes argument are the same.

- **regex_code_structure: regular expression defining the code structure of**

  transformer codes

- regex_template_prepend: passed to SISEPUEDEFileStructure()
- **kwargs

> **_trfunc_agrc_improve_rice_management**(*df_input: DataFrame | None = None, magnitude: float = 0.45, strat: int | None = None, vec_implementation_ramp: ndarray | Dict[str, int] | None = None*)→ DataFrame

Implement the "Improve Rice Management" AGRC transformer on input DataFrame df_input.

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Minimum target fraction of rice production under improved management.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

Transformer Docstrings

Note that parameters shown in the docstrings below are keyword arguments. In general, the user should **not** set a value for the `strat` keyword argument.

---

transformers.Transformers.agrc_decrease_exports(**args, **kwargs*)→ Any

Implement the "Decrease Exports" AGRC transformer on input DataFrame df_input (reduce by 50%)

Transformation Code :

TFR:AGRC:DEC_EXPORTS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of decrease in exports. If using the default value of *magnitude_type == "scalar"*, this magnitude will scale the final time value downward by this factor. NOTE: If magnitude_type changes, then the behavior of the trasnformation will change.
- **magnitude_type** (*str*) – Type of magnitude, as specified in *transformers.lib.general.transformations_general*. See *? transformers.lib.general.transformations_general* for more information on the specification of magnitude_type for general transformer values.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.agrc_expand_conservation_agriculture(**args, **kwargs*) → Any

Implement the "Expand Conservation Agriculture" AGRC transformer on input DataFrame df_input.

NOTE: Sets a new floor for F_MG (as described in in V4 Equation 2.25 (2019R)) to reduce losses of soil organic carbon through no-till in cropland + reduces removals and burning of crop residues, increasing residue covers on fields.

Transformation Code :

TFR:AGRC:INC_CONSERVATION_AGRICULTURE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_categories_to_magnitude** (*Union[Dict[str, float], None]*) –

Conservation agriculture is practically applied to only select crop types. Use the dictionary to map SISEPUEDE crop categories to target implementation magnitudes. * If None, maps to the following dictionary:

{

"cereals": 0.8, "fibers": 0.8, "other_annual": 0.8, "pulses": 0.5, "tubers": 0.5, "vegetables_and_vines": 0.5,

}

- **magnitude_burned** (*float*) – Target fraction of residues that are burned
- **magnitude_removed** (*float*) – Maximum fraction of residues that are removed
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.agrc_improve_rice_management(**args*, ***kwargs*) → Any

Implement the "Improve Rice Management" AGRC transformer on input DataFrame df_input.

Transformation Code :

TFR:AGRC:DEC_CH4_RICE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Minimum target fraction of rice production under improved management.
- **strat** (*int*) – Optional strategy value to specify for the transformation

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.agrc_increase_crop_productivity(**args*, ***kwargs*) → Any

Implement the "Increase Crop Productivity" AGRC transformer on input DataFrame df_input.

Transformation Code :

TFR:AGRC:INC_PRODUCTIVITY

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*Union[float, Dict[str, float]]*) – Magnitude of productivity increase to apply to crops (e.g., a 20% increase is entered as 0.2); can be specified as one of the following * float: apply a single scalar increase to productivity for all crops * dict: specify crop productivity increases individually, with each key being a crop and the associated value being the productivity increase
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.agrc_reduce_supply_chain_losses(**args*, ***kwargs*) → Any

Implement the "Reduce Supply Chain Losses" AGRC transformer on input DataFrame df_input.

Transformation Code :

TFR:AGRC:DEC_LOSSES_SUPPLY_CHAIN

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of reduction in supply chain losses. Specified as a fraction (e.g., a 30% reduction is specified as 0.3)
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.baseline(**args, **kwargs*)→ Any

Create a return function to support the baseline transformation

Function Arguments

keyword - df_input:
kwtype - df_input: optional input dataframe
keyword - strat:
kwtype - strat: optional strategy id to pass

---

transformers.Transformers.ccsq_increase_air_capture(**args, **kwargs*)→ Any

Implement the “Increase Direct Air Capture” CCSQ transformer on input DataFrame df_input

Transformation Code :

TFR:CCSQ:INC_CAPTURE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – final total, in MT, of direct air capture capacity installed at 100% implementation
- **strat** (*int*) – Optional strategy value to specify for the transformation

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.entc_clean_hydrogen(**args, **kwargs*)→ Any

Implement "green hydrogen" transformer requirements by forcing at least 95% (or *magnitude*) of hydrogen production to come from electrolysis.

Transformation Code :

TFR:ENTC:TARGET_CLEAN_HYDROGEN

Parameters:

- **categories_source** (*Union[List[str], None]*) – Hydrogen-producing technology categories that are reduced in response to increases in green hydrogen. * If None, defaults to
  [
  "fp_hydrogen_gasification", "fp_hydrogen_reformation", "fp_hydrogen_reformation_ccs"
  ]
- **categories_target** (*Union[List[str], None]*) – Hydrogen-producing technology categories that are considered green; they will produce *magnitude* of hydrogen by 100% implementation. * If None, defaults to
  [
  "fp_hydrogen_electrolysis"
  ]
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – Target fraction of hydrogen from clean (categories_source) sources. In general, this is 95% from electrolysis.

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.entc_least_cost(**args, **kwargs*)→ Any

Implement the "Least Cost" ENTC transformer on input DataFrame df_input

Transformation Code :

TFR:ENTC:LEAST_COST_SOLUTION

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.entc_reduce_transmission_losses(**args, **kwargs*)→ Any

Implement the "Reduce Transmission Losses" ENTC transformer on input DataFrame df_input

Transformation Code :

TFR:ENTC:DEC_LOSSES

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – magnitude of transmission loss in final time; behavior depends on *magnitude_type*
- **magnitude_type** (*str*) –
  - scalar (if *magnitude_type == "basline_scalar"*)

  - final value (if *magnitude_type == "final_value"*)
  - final value ceiling (if *magnitude_type == "final_value_ceiling"*)

- **min_loss** (*float*) – Minimum feasible transmission loss in the system
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.entc_renewable_electricity`(**args*, ***kwargs*) → Any

Implement the "renewables target" transformer (shared repeatability), which sets a Minimum Share of Production from renewable energy as a target.

Transformation Code :

TFR:ENTC:TARGET_RENEWABLE_ELEC

Parameters:

- **categories_entc_max_investment_ramp** (*Union[List[str], None]*) – Categories to cap investments in
- **categories_entc_renewable** (*Union[List[str], None]*) – Power plant technologies to consider as renewable energy sources
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_entc_renewable_target_msp** (*Union[Dict[str, float], None]*) – Optional dictionary mapping renewable ENTC categories to MSP fractions. Can be used to ensure some minimum contribution of certain renewables–e.g.,

  {

  "pp_hydropower": 0.1, "pp_solar": 0.15

  }

  will ensure that hydropower is at least 10% of the mix and solar is at least 15%.

- **magnitude** (*float*) – Minimum target fraction of electricity produced from renewable sources by 100% implementation
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.fgtv_maximize_flaring(**args*, ***kwargs*) → Any

Implement the "Maximize Flaring" FGTV transformer on input DataFrame df_input

Transformation Code :

TFR:FGTV:INC_FLARE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fraction of vented methane that is flared.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.fgtv_minimize_leaks(**args*, ***kwargs*) → Any

Implement the "Minimize Leaks" FGTV transformer on input DataFrame df_input

Transformation Code :

TFR:FGTV:DEC_LEAKS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fraction of leaky sources (pipelines, storage, etc) that are fixed

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.inen_fuel_switch_heat(**args, **kwargs*)→ Any

Implement the "Fuel switch low-temp thermal processes to industrial heat pumps" or/and "Fuel switch medium and high-temp thermal processes to hydrogen and electricity" INEN transformations on input DataFrame df_input (note: these must be combined in a new function instead of as a composition due to the electricity shift in high-heat categories)

Transformation Code :

TFR:INEN:SHIFT_FUEL_HEAT

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **frac_high_given_high** (*Union[float, dict, None]*) – In high heat categories, fraction of heat demand that is high (NOTE: needs to be switched to per industry). * If specified as a float, this is applied to all high heat categories * If specified as None, uses the following dictionary as a default:

  (TEMP: from https://www.sciencedirect.com/science/article/pii/S0360544222018175?via%3Dihub#bib34 [see sainz_et_al_2022])

  {

  "cement": 0.88, # use non-metallic minerals "chemicals": 0.5, "glass": 0.88, "lime_and_carbonite": 0.88, "metals": 0.92, "paper": 0.18,

  }

- **frac_switchable** (*float*) – Fraction of demand that can be switched

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.inen_maximize_energy_efficiency(**args, **kwargs*)→ Any

Implement the "Maximize Industrial Energy Efficiency" INEN transformer on input DataFrame df_input

Transformation Code :

TFR:INEN:INC_EFFICIENCY_ENERGY

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of energy efficiency increase (applied to industrial efficiency factor)
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.inen_maximize_production_efficiency(**args, **kwargs*)→ Any

Implement the "Maximize Industrial Production Efficiency" INEN transformer on input DataFrame df_input

Transformation Code :

TFR:INEN:INC_EFFICIENCY_PRODUCTION

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify

- **magnitude** (*float*) – Magnitude of energy efficiency increase (applied to industrial production factor)
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_demand_managment(**args, **kwargs*)→ Any

Implement the "Demand Management" IPPU transformer on input DataFrame df_input. Reduces industrial production.

Transformation Code :

TFR:IPPU:DEC_DEMAND

Parameters:
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional reduction in demand in accordance with vec_implementation_ramp
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_reduce_cement_clinker(**args, **kwargs*)→ Any

Implement the "Reduce cement clinker" IPPU transformer on input DataFrame df_input. Implements a cap on the fraction of cement that is produced using clinker (magnitude)

Transformation Code :

TFR:IPPU:DEC_CLINKER

Parameters:
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify

- **magnitude** (*float*) – fraction of cement producd using clinker
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_reduce_hfcs(**args*, ***kwargs*) → Any

Implement the "Reduces HFCs" IPPU transformer on input DataFrame df_input

Transformation Code :

TFR:IPPU:DEC_HFCS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional reduction in HFC emissions in accordance with vec_implementation_ramp
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_reduce_n2o(**args*, ***kwargs*) → Any

Implement the "Reduces N2O" IPPU transformer on input DataFrame df_input

Transformation Code :

TFR:IPPU:DEC_N2O

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional reduction in IPPU N2O emissions in accordance with vec_implementation_ramp

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_reduce_other_fcs(**args*, ***kwargs*) → Any

Implement the "Reduces Other FCs" IPPU transformer on input DataFrame df_input

Transformation Code :

TFR:IPPU:DEC_OTHER_FCS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – Fractional reduction in IPPU other FC emissions in accordance with vec_implementation_ramp
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.ippu_reduce_pfcs(**args*, ***kwargs*) → Any

Implement the "Reduces Other FCs" IPPU transformer on input DataFrame df_input

Transformation Code :

TFR:IPPU:DEC_PFCS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional reduction in IPPU other FC emissions in accordance with vec_implementation_ramp

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lndu_expand_silvopasture(**args, **kwargs*)→ Any

Increase the use of silvopasture by shifting pastures to secondary forest.

NOTE: This transformer relies on modifying transition matrices, which can compound some minor numerical errors in the crude implementation taken here. Final area prevalences may not reflect get_matrix_column_scalarget_matrix_column_scalarprecise shifts.

Transformation Code :

TFR:LNDU:INC_SILVOPASTURE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – Magnitude of increase in fraction of pastures subject to silvopasture
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lndu_expand_sustainable_grazing(**args, **kwargs*)→ Any

Implement the “Expand Sustainable Grazing” LNDU transformer on input DataFrame df_input.

NOTE: Sets a new floor for F_MG (as described in in V4 Equation 2.25 (2019R)) through improved grassland management (grasslands).

Transformation Code :

TFR:LNDU:DEC_SOC_LOSS_PASTURES

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fraction of pastures subject to improved pasture management. This value acts as a floor, so that if the existing value is greater than is specified by the transformation, the existing value will be maintained.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ram** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.lndu_increase_reforestation`(**args*, ***kwargs*) → Any

Implement the “Increase Reforestation” FRST transformer on input DataFrame df_input.

Transformation Code :

TFR:LNDU:INC_REFORESTATION

Parameters:

- **cats_inflow_restriction** (*Union[List[str], None]*) – LNDU categories to allow to transition into secondary forest; don’t specify categories that cannot be reforested
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional increase in secondary forest area to specify; for example, a 10% increase in secondary forests is specified as 0.1
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp

that starts at the time specified in the configuration.

---

transformers.Transformers.lndu_partial_reallocation(**args*, ***kwargs*)→ Any

Implement land use reallocation factor as a ramp.

Transformation Code :

TFR:LNDU:PLUR

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **force** (*bool*) – If the baseline includes LURF > 0, then this transformer will not work by default to avoid double-implementation. Set force = True to force the transformer to further modify the LURF
- **magnitude** (*float*) – Land use reallocation factor value with implementation ramp vector
- **strat** (*int*) – Optional strategy index to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray*, *Dict[str*, *int]*, *None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lndu_stop_deforestation(**args*, ***kwargs*)→ Any

Implement the “Stop Deforestation” FRST transformer on input DataFrame df_input.

Transformation Code :

TFR:LNDU:DEC_DEFORESTATION

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of final primary forest transition probability into itself; higher magnitudes indicate less deforestation.

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lsmm_improve_manure_management_cattle_pigs(**args, **kwargs*)→ Any

Implement the "Improve Livestock Manure Management for Cattle and Pigs" transformer on the input DataFrame.

Transformation Code :

TFR:LSMM:INC_MANAGEMENT_CATTLE_PIGS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_lsmm_pathways** –
  Dictionary allocating treatment to LSMM categories as fractional targets (must sum to <= 1). If None, defaults to
  dict_lsmm_pathways = {

  "anaerobic_digester": 0.59375, # 0.625*0.95, "composting": 0.11875, # 0.125*0.95, "daily_spread": 0.2375, # 0.25*0.95,

  }

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_cats_lvst** (*Union[List[str], None]*) –
  LVST categories receiving treatment in this transformation. Default (if None) is
  [

  "cattle_dairy", "cattle_nondairy", "pigs"

  ]

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lsmm_improve_manure_management_other(**args, **kwargs*)→ Any

Implement the "Improve Livestock Manure Management for Other Animals" LSMM transformer on the input DataFrame.

Transformation Code :

TFR:LSMM:INC_MANAGEMENT_OTHER

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_lsmm_pathways** (*Union[dict, None]*) – Dictionary allocating treatment to LSMM categories as fractional targets (must sum to <= 1). If None, defaults to
  dict_lsmm_pathways = {

  "anaerobic_digester": 0.475, # 0.5*0.95, "composting": 0.2375, # 0.25*0.95, "dry_lot": 0.11875, # 0.125*0.95, "daily_spread": 0.11875, # 0.125*0.95,

  }

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_cats_lvst** (*Union[List[str], None]*) – LVST categories receiving treatment in this transformation. Default (if None) is [
  "buffalo", "goats", "horses", "mules", "sheep",
  ]

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

transformers.Transformers.lsmm_improve_manure_management_poultry(**args*, ***kwargs*)→ Any

Implement the “Improve Livestock Manure Management for Poultry” LSMM transformer on the input DataFrame.

Transformation Code :

TFR:LSMM:INC_MANAGEMENT_POULTRY

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_lsmm_pathways** (*Union[dict, None]*) – Dictionary allocating treatment to LSMM categories as fractional targets (must sum to <= 1). If None, defaults to
  dict_lsmm_pathways = {
  ”anaerobic_digester”: 0.475, # 0.5*0.95, “poultry_manure”: 0.475, # 0.5*0.95,
  }
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_cats_lvst** (*Union[List[str], None]*) – LVST categories receiving treatment in this transformation. Default (if None) is [
  ”chickens”,
  ]
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

transformers.Transformers.lsmm_increase_biogas_capture(**args*, ***kwargs*)→ Any

Implement the “Increase Biogas Capture at Anaerobic Decomposition Facilities” transformer on the input DataFrame.

Transformation Code :

TFR:LSMM:INC_CAPTURE_BIOGAS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – Target minimum fraction of biogas that is captured atanerobic decomposition facilities.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lvst_decrease_exports(**args, **kwargs*)→ Any

Implement the “Decrease Exports” LVST transformer on input DataFrame df_input (reduce by 50%)

Transformation Code :

TFR:LVST:DEC_EXPORTS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional reduction in exports applied directly to time periods (reaches 100% implementation when ramp reaches 1)
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.lvst_increase_productivity(**args, **kwargs*)→ Any

Implement the “Increase Livestock Productivity” LVST transformer on

input DataFrame df_input.

Transformation Code :

TFR:LVST:INC_PRODUCTIVITY

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Fractional increase in productivity applied to carrying capcity for livestock
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.lvst_reduce_enteric_fermentation`(**args*, ***kwargs*) → Any

Implement the “Reduce Enteric Fermentation” LVST transformer on input DataFrame df_input.

Transformation Code :

TFR:LVST:DEC_ENTERIC_FERMENTATION

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_lvst_reductions** (*Union[dict, None]*) – Dictionary allocating mapping livestock categories to associated reductions in enteric fermentation. If None, defaults to
  {

  “buffalo”: 0.4, “cattle_dairy”: 0.4, “cattle_nondairy”: 0.4, “goats”: 0.56, “sheep”: 0.56

  }
- **strat** (*int*) – Optional strategy value to specify for the transformation

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.pflo_industrial_ccs(**args, **kwargs*)→ Any

Implement the "Industrial Point of Capture" transformer on input DataFrame df_input (affects IPPU and INEN).

Transformation Code :

TFR:PFLO:INC_IND_CCS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.plfo_healthier_diets(**args, **kwargs*)→ Any

Implement the "Healthier Diets" transformer on input DataFrame df_input (affects GNRL and and AGRC [NOTE: AGRC component currently not implemented]).

Transformation Code :

TFR:PFLO:INC_HEALTHIER_DIETS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude_red_meat** (*float*) – Final period maximum fraction of per capita red meat consumption relative to baseline (e.g., 0.5 means that people eat 50% as much red meat as they would have without the intervention)

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.scoe_fuel_switch_electrify(**args*, ***kwargs*) → Any

Implement the "Switch to electricity for heat using heat pumps, electric stoves, etc." INEN transformer on input DataFrame df_input

Transformation Code :

TFR:SCOE:SHIFT_FUEL_HEAT

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magntiude of fraction of heat energy that is electrified
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.scoe_increase_applicance_efficiency(**args*, ***kwargs*) → Any

Implement the "Increase appliance efficiency" SCOE transformer on input DataFrame df_input

Transformation Code :

TFR:SCOE:INC_EFFICIENCY_APPLIANCE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify

- **magnitude** (*float*) – Fractional increase in applieance energy efficiency
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.scoe_reduce_heat_energy_demand(**args*, ***kwargs*)→ Any

Implement the "Reduce end-use demand for heat energy by improving building shell" SCOE transformer on input DataFrame df_input

Transformation Code :

TFR:SCOE:DEC_DEMAND_HEAT

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Reduction in heat energy demand, driven by retrofitting and changes in use
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.soil_reduce_excess_fertilizer(**args*, ***kwargs*)→ Any

Implement the "Reduce Excess Fertilizer" SOIL transformer on input DataFrame df_input.

Transformation Code :

TFR:SOIL:DEC_N_APPLIED

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – fractional reduction in fertilier N to achieve in accordane with vec_implementation_ramp
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.soil_reduce_excess_liming(**args*, ***kwargs*) → Any

Implement the “Reduce Excess Liming” SOIL transformer on input DataFrame df_input.

Transformation Code :

TFR:SOIL:DEC_LIME_APPLIED

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – fractional reduction in lime application to achieve in accordane with vec_implementation_ramp
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trde_reduce_demand(**args*, ***kwargs*) → Any

Implement the “Reduce Demand” TRDE transformer on input DataFrame

df_input

Transformation Code :

TFR:TRDE:DEC_DEMAND

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **e** (*magnitud*) – Fractional reduction in transportation demand applied to all transportation categories.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_electrify_light_duty_road(**args*, ***kwargs*)→ Any

Implement the "Electrify Light-Duty" TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:SHIFT_FUEL_LIGHT_DUTY

Parameters:

- **categories** (*List[str]*) – Transportation categories to include; defaults to "road_light"
- **dict_fuel_allocation** (*Union[dict, None]*) –
  Optional dictionary defining fractional allocation of fuels in fuel switch. If undefined, defaults to

  {

  "fuel_electricity": 1.0

  }

  NOTE: keys must be valid TRNS fuels and values in the dictionary must sum to 1.
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – fraction of light duty vehicles electrified
- **strat** (*int*) – Optional strategy value to specify for the transformation

- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_electrify_rail(**args, **kwargs*)→ Any

Implement the "Electrify Rail" TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:SHIFT_FUEL_RAIL

Parameters:

- **categories** (*List[str]*) – Transportation categories to include; defaults to ["rail_freight", "rail_passenger"]
- **dict_fuel_allocation** (*Union[dict, None]*) – **Optional dictionary dictionary defining fractional allocation of fuels in fuel switch. If undefined, defaults to**

  {

  "fuel_electricity": 1.0

  }

  NOTE: keys must be valid TRNS fuels and values in the dictionary must sum to 1.

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – fraction of light duty vehicles electrified
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_fuel_switch_maritime(**args, **kwargs*)→ Any

Implement the "Fuel-Swich Maritime" TRNS transformer on input DataFrame df_input.

By default, transfers mangitude to hydrogen from gasoline and diesel; e.g., with magnitude = 0.7, then 70% of diesel and gas demand are transfered to fuels in fuels_target. The rest of the fuel demand is then transferred to electricity.

Transformation Code :

TFR:TRNS:SHIFT_FUEL_MARITIME

Parameters:

- **categories** (*List[str]*) – Transportation categories to include; defaults to ["water_borne"]
- **dict_allocation_fuels_target** (*Union[dict, None]*) – Optional dictionary allocating target fuels. If None, defaults to {
  "fuel_hydrogen": 1.0,
  }
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **fuels_source** (*List[str]*) – Fuels to transfer out; for F the percentage of TRNS demand met by fuels in fuels source, M*F (M = magtnitude) is transferred to fuels defined in dict_allocation_fuels_target
- **magnitude** (*float*) – Fraction of fuels_source (gas and diesel, e.g.) that is shifted to target fuels fuels_target (hydrogen is default, can include ammonia) for categories. Note, remaining is shifted to electricity
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_fuel_switch_medium_duty_road(**args*, ***kwargs*)→ Any

Implement the "Fuel-Switch Medium Duty" TRNS transformer on input DataFrame df_input. By default, transfers mangitude to electricity from gasoline and diesel; e.g., with magnitude = 0.7, then 70% of diesel and gas demand are transfered to fuels in fuels_target. The rest of the fuel demand is then transferred to hydrogen.

Transformation Code :

TFR:TRNS:SHIFT_FUEL_MEDIUM_DUTY

Parameters:

- **categories** (*List[str]*) –
  Transportation categories to include; defaults to [
  "road_heavy_freight", "road_heavy_regional", "public"
  ]
- **dict_allocation_fuels_target** (*Union[dict, None]*) –
  Optional dictionary allocating target fuels. If None, defaults to {
  "fuel_electricity": 1.0,
  }
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **fuels_source** (*List[str]*) – Fuels to transfer out; for F the percentage of TRNS demand met by fuels in fuels source, M*F (M = magtnitude) is transferred to fuels defined in dict_allocation_fuels_target
- **magnitude** (*float*) – Fraction of fuels_source (gas and diesel, e.g.) that shifted to target fuels fuels_target (hydrogen is default, can include ammonia). Note, remaining is shifted to electricity
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_increase_efficiency_electric(**args*, ***kwargs*)→ Any

Implement the "Increase Electric Efficiency" TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:INC_EFFICIENCY_ELECTRIC

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Increase the efficiency of electric vehicales by this proportion
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_increase_efficiency_non_electric(**args, **kwargs*)→ Any

Implement the “Increase Non-Electric Efficiency” TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:INC_EFFICIENCY_NON_ELECTRIC

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Increase the efficiency of non-electric vehicales by this proportion
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_increase_occupancy_light_duty(**args, **kwargs*)→ Any

Implement the “Increase Vehicle Occupancy” TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:INC_OCCUPANCY_LIGHT_DUTY

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Increase the occupancy rate of light duty vehicles by this proporiton
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.trns_mode_shift_freight`(**args*, ***kwargs*) → Any

Implement the "Mode Shift Freight" TRNS transformer on input DataFrame df_input. By Default, transfer 20% of aviation and road heavy freight to rail freight.

Transformation Code :

TFR:TRNS:SHIFT_MODE_FREIGHT

Parameters:

- **categories_out** (*List[str]*) – Categories to shift out of
- **dict_categories_target** (*Union[Dict[str, float], None]*) – Dictionary mapping target categories to proportional allocation of mode mass. If None, defaults to {
  "rail_freight": 1.0
  }
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of mode mass to shift out of cats_out
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp

that starts at the time specified in the configuration.

---

transformers.Transformers.trns_mode_shift_public_private(**args*, ***kwargs*) → Any

Implement the "Mode Shift Passenger Vehicles to Others" TRNS

transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:SHIFT_MODE_PASSENGER

Parameters:

- **categories_out** (*List[str]*) – Categories to shift out of
- **dict_categories_target** (*Union[Dict[str, float], None]*) – Dictionary mapping target categories to proportional allocation of mode mass. If None, defaults to {
  "human_powered": (1/6), "powered_bikes": (2/6), "public": 0.5
  }
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** (*float*) – Magnitude of mode mass to shift out of cats_out
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trns_mode_shift_regional(**args*, ***kwargs*) → Any

Implement the "Mode Shift Regional Travel" TRNS transformer on input DataFrame df_input

Transformation Code :

TFR:TRNS:SHIFT_MODE_REGIONAL

Parameters:

- **dict_categories_out** (*Dict[str, float]*) – Dictionary mapping

categories to shift out of to the magnitude of the outward shift
- **dict_categories_target** (*Union[Dict[str, float], None]*) – Dictionary mapping target categories to proportional allocation of mode mass. If None, defaults to {
  "road_heavy_regional": 1.0
  }
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.trww_increase_biogas_capture`(**args*, ***kwargs*) → Any

Implement the "Increase Biogas Capture at Wastewater Treatment Plants"

TRWW transformer on input DataFrame df_input

Transformation Code :

TFR:TRWW:INC_CAPTURE_BIOGAS

Parameters:
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – final magnitude of biogas capture at TRWW facilties. NOTE: If specified as a float, the same value applies to both

landfill and biogas. Specify as a dictionary to specifiy different capture fractions by TRWW technology, e.g.,

magnitude = {

"treated_advanced_anaerobic": 0.85,
"treated_secondary_anaerobic": 0.5

}

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.trww_increase_septic_compliance(**args, **kwargs*)→ Any

Implement the "Increase Compliance" TRWW transformer on input DataFrame df_input

Transformation Code :

TFR:TRWW:INC_COMPLIANCE_SEPTIC

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – final magnitude of compliance at septic tanks that are installed
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.wali_improve_sanitation_industrial(**args, **kwargs*)→ Any

Implement the “Improve Industrial Sanitation” WALI transformer on input DataFrame df_input

Transformation Code :

TFR:WALI:INC_TREATMENT_INDUSTRIAL

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_magnitude** (*Union[Dict[str, float], None]*) – Target allocation, across TRWW (Wastewater Treatment) categories (categories are keys), of treatment as total fraction. * E.g., to acheive 80% of treatment from advanced anaerobic and 10% from scondary aerobic by the final time period, the following dictionary would be specified:
  dict_magnitude = {

  “treated_advanced_anaerobic”: 0.8,
  “treated_secondary_anaerobic”: 0.1

  }
  If None, defaults to:
  {

  “treated_advanced_anaerobic”: 0.8,
  “treated_secondary_aerobic”: 0.1,
  “treated_secondary_anaerobic”: 0.1,

  }

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.wali_improve_sanitation_rural(**args*, ***kwargs*) → Any

Implement the “Improve Rural Sanitation” WALI transformer on input DataFrame df_input

Transformation Code :

TFR:WALI:INC_TREATMENT_RURAL

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_magnitude** (*Union[Dict[str, float], None]*) – Target allocation, across TRWW (Wastewater Treatment) categories (categories are keys), of treatment as total fraction. * E.g., to acheive 80% of treatment from advanced anaerobic and 10% from scondary aerobic by the final time period, the following dictionary would be specified:
  dict_magnitude = {

  “treated_advanced_anaerobic”: 0.8,
  “treated_secondary_anaerobic”: 0.1

  }
  If None, defaults to:
  {

  “treated_septic”: 1.0,

  }
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.wali_improve_sanitation_urban`(**args*, ***kwargs*) → Any

Implement the “Improve Urban Sanitation” WALI transformer on input DataFrame df_input

Transformation Code :

TFR:WALI:INC_TREATMENT_URBAN

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **dict_magnitude** (*Union[Dict[str, float], None]*) –
  Target allocation, across TRWW (Wastewater Treatment) categories (categories are keys), of treatment as total fraction. * E.g., to acheive 80% of treatment from advanced anaerobic and 10% from scondary aerobic by the final time period, the following dictionary would be specified:
  dict_magnitude = {

  “treated_advanced_anaerobic”: 0.8,
  “treated_secondary_anaerobic”: 0.1

  }
  If None, defaults to:
  {

  “treated_advanced_aerobic”: 0.3,
  “treated_advanced_anaerobic”: 0.3,
  “treated_secondary_aerobic”: 0.2,
  “treated_secondary_anaerobic”: 0.2,

  }

- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.waso_descrease_consumer_food_waste(**args*, ***kwargs*) → Any

Implement the “Decrease Municipal Solid Waste” WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:DEC_CONSUMER_FOOD_WASTE

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – reduction in food waste sent to munipal solid waste treatment stream
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.waso_energy_from_biogas(**args*, ***kwargs*) → Any

Implement the “Energy from Captured Biogas” WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:INC_ENERGY_FROM_BIOGAS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – final magnitude of energy use from captured biogas.
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.waso_energy_from_incineration(**args*, ***kwargs*) → Any

Implement the “Energy from Solid Waste Incineration” WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:INC_ENERGY_FROM_INCINERATION

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – final magnitude of waste that is incinerated that is recovered for energy use
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

---

transformers.Transformers.waso_increase_anaerobic_treatment_and_composting(**args*, ***kwargs*)→ Any

Implement the "Increase Anaerobic Treatment and Composting" WASO

transformer on input DataFrame df_input.

Note that 0 <= magnitude_biogas + magnitude_compost should be <= 1;

if they exceed 1, they will be scaled proportionally to sum to 1

Transformation Code :

TFR:WASO:INC_ANAEROBIC_AND_COMPOST

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude_biogas** (-) – using anaerobic treatment
- **magnitude_compost** (-) – using compost
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

transformers.Transformers.waso_increase_biogas_capture(**args*, ***kwargs*)→ Any

Implement the "Increase Biogas Capture at Anaerobic Treatment Facilities

and Landfills" WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:INC_CAPTURE_BIOGAS

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** –
  final magnitude of biogas capture at landfill and anaerobic digestion facilties. NOTE: If specified as a float, the same value applies to both
  landfill and biogas. Specify as a dictionary to specifiy different capture fractions by WASO technology, e.g.,

  magnitude = {

  "landfill": 0.85, "biogas": 0.5

  }
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray*, *Dict[str*, *int]*, *None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

transformers.Transformers.waso_increase_landfilling(**args*, ***kwargs*)→ Any

Implement the "Increase Landfilling" WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:INC_LANDFILLING

Parameters:

- **df_input** (*pd.DataFrame*) – Optional data frame containing

trajectories to modify
- **magnitude** (*float*) – fraction of non-recycled solid waste (including composting and anaerobic digestion) sent to landfills
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

`transformers.Transformers.waso_increase_recycling`(**args*, ***kwargs*)→ Any

Implement the "Increase Recycling" WASO transformer on input DataFrame df_input

Transformation Code :

TFR:WASO:INC_RECYCLING

Parameters:
- **df_input** (*pd.DataFrame*) – Optional data frame containing trajectories to modify
- **magnitude** – magnitude of recylables that are recycled
- **strat** (*int*) – Optional strategy value to specify for the transformation
- **vec_implementation_ramp** (*Union[np.ndarray, Dict[str, int], None]*) – Optional vector or dictionary specifying the implementation scalar ramp for the transformation. If None, defaults to a uniform ramp that starts at the time specified in the configuration.

magnitude_type

- *baseline_additive*: add the magnitude to the baseline
- *baseline_scalar*: multiply baseline value by magnitude
- ***baseline_scalar_diff_reduction*: reduce the difference between**

  the value in the baseline time period and the upper bound (NOTE: requires specification of bounds to work) by magnitude

- ***final_value*: magnitude is the final value for the variable to**

take (achieved in accordance with vec_ramp)

- ***final_value_ceiling*: magnitude is the lesser of (a) the existing**

  final value for the variable to take (achieved in accordance with vec_ramp) or (b) the existing specified final value, whichever is smaller

- ***final_value_floor*: magnitude is the greater of (a) the existing**

  final value for the variable to take (achieved in accordance with vec_ramp) or (b) the existing specified final value, whichever is greater

- ***transfer_value*: transfer value from categories to other**

  categories. Must specify “categories_source” & “categories_target” in dict_modvar_specs. See description below in OPTIONAL for information on specifying this.

- ***transfer_scalar_value*: transfer value from categories to other**

  categories based on a scalar. Must specify “categories_source” & “categories_target” in dict_modvar_specs. See description below in OPTIONAL for information on specifying this.

- ***transfer_value_to_acheieve_magnitude*: transfer value from**

  categories to other categories to acheive a target magnitude. Must specify “categories_source” & “categories_target” in dict_modvar_specs. See description below in OPTIONAL for information on specifying this.

- *vector_specification*: simply enter a vector to use for region

vec_implementation_ramp

The implementation ramp vector is a vector that defines the fractional implementation of a policy over time periods.

- *n_tp_ramp*: Number of time periods it takes for the intervention to reach full effect. If not specified
- *tp_0_ramp*: Final time period with 0 change from baseline
- *a*: sigmoid magnitude parameter; set to 0 for linear, 1 for full sigmoid
- *b*: linear coefficient; set to 2 for linear (div by 2) or 0 for sigmoid

- *c*: denominator exponee–in linear, set to 1 (adds term 1 + 1 to denominator); for sigmoid, set to np.e
- *d* (optional): centroid for sigmoid/linear function. If using a sigmoid, this is the position of 0.5 in years $\geq r_0$

Linear vector set a = 0, b = 2, c = 1, d = r_0 + (n - r_0 - r_1)/2

Sigmoid: set a = 1, b = 0, c = math.e, d = r_0 + (n - r_0 - r_1)/2